

Chapter 1: Data Structures

Data structures are specialized formats for organizing, processing, retrieving, and storing data. They provide a means to manage large amounts of data efficiently for uses such as large databases and internet indexing services.

An array is a collection of elements stored at contiguous memory locations. Arrays allow random access to elements using an index. The time complexity for accessing an element by index is $O(1)$, while searching for an element takes $O(n)$ in the worst case.

A linked list is a linear data structure where elements are stored in nodes. Each node contains data and a reference (pointer) to the next node. Unlike arrays, linked lists do not store elements in contiguous memory locations. Insertion and deletion at the beginning of a linked list takes $O(1)$ time.

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. The main operations are push (add an element to the top), pop (remove the top element), and peek (view the top element without removing it). Stacks are used in function call management, expression evaluation, and backtracking algorithms.

A queue is a linear data structure that follows the First In First Out (FIFO) principle. The main operations are enqueue (add an element to the rear) and dequeue (remove an element from the front). Queues are used in scheduling, buffering, and breadth-first search.

A binary tree is a hierarchical data structure where each node has at most two children, called the left child and right child. A Binary Search Tree (BST) is a binary tree where the left subtree contains nodes with values less than the root, and the right subtree contains nodes with values greater than the root. The average time complexity for search, insert, and delete in a BST is $O(\log n)$.

A hash table is a data structure that maps keys to values using a hash function. The hash function converts a key into an index in an array. Hash tables provide $O(1)$ average time complexity for search, insert, and delete operations. Collision resolution techniques include chaining and open addressing.

Chapter 2: Algorithms

An algorithm is a step-by-step procedure for solving a problem or accomplishing a task. Algorithms are evaluated based on their time complexity and space complexity.

Sorting algorithms arrange elements in a specific order. Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Its time complexity is $O(n^2)$. Merge Sort uses the divide-and-conquer strategy, dividing the array into halves, sorting each half, and merging them. Merge Sort has a time complexity of $O(n \log n)$ and is stable.

Quick Sort also uses divide-and-conquer. It selects a pivot element and partitions the array around the pivot. Elements smaller than the pivot go to the left, and larger ones go to the right. Quick Sort has an average time complexity of $O(n \log n)$ but a worst-case of $O(n^2)$.

Binary Search is an efficient algorithm for finding an element in a sorted array. It works by repeatedly dividing the search interval in half. If the target value is less than the middle element, the search continues in the lower half. Binary Search has a time complexity of $O(\log n)$.

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a stack (or recursion). DFS is used for topological sorting, detecting cycles, and solving maze problems.

Breadth-First Search (BFS) is a graph traversal algorithm that explores all neighbors at the current depth before moving to the next level. It uses a queue. BFS finds the shortest path in an unweighted graph.

Dynamic Programming (DP) is a technique for solving problems by breaking them into overlapping subproblems and storing their solutions. The two approaches are top-down (memoization) and bottom-up (tabulation). Classic DP problems include the Fibonacci sequence, knapsack problem, and longest common subsequence.